ورود به دنیای هوش‌مصنوعی

دوره‌ی مقدماتی و رایگان برای عموم علاقه‌مندان

جلسه‌ی هشتم: بینایی ماشین، چشم‌های هوش‌مصنوعی

علی شریفی زارچی

رئیس کمیته‌ی علمی بین‌المللی المپیاد جهانی هوش‌مصنوعی

عضو کمیته‌ی علمی بین‌المللی المپیاد جهانی کامپیوتر

مرور جلسه‌ی قبل

- نورون مصنوعی: جمع وزن‌دار + تابع فعال‌سازی
- شبکه‌ی عصبی: لایه‌ی ورودی، لایه‌های پنهان، لایه‌ی خروجی
- هر لایه یک سطح از انتزاع: از پیکسل تا مفهوم
- آموزش: پس‌انتشار خطا + گرادیان کاهشی (کوه‌نورد!)
- امروز: شبکه‌هایی که «می‌بینند»

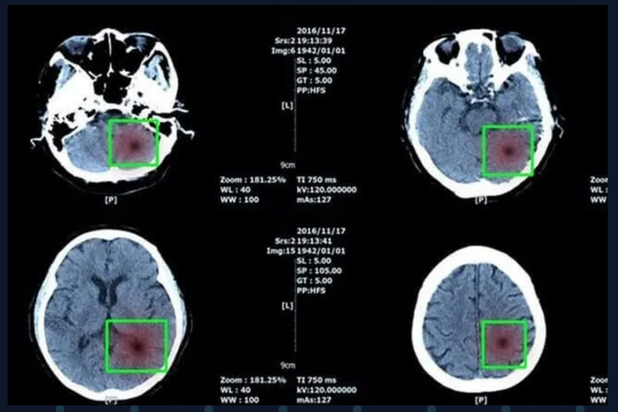
بینایی ماشین

پزشکی

تشخیص تومور در MRI

غربالگری شبکیه‌ی چشم

شمارش سلول‌ها

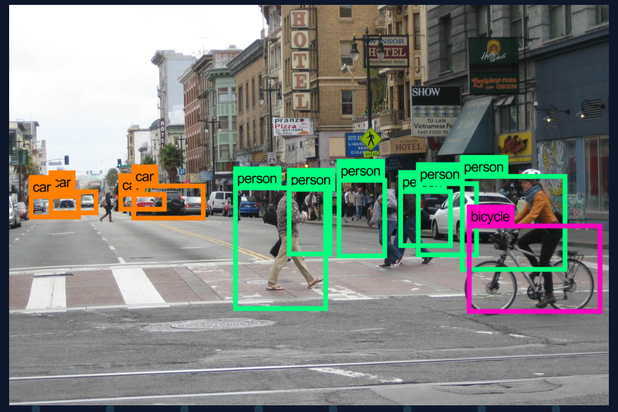


حمل‌ونقل

خودروی خودران

پلاک‌خوان و کنترل ترافیک

عیب‌یابی جاده و پل

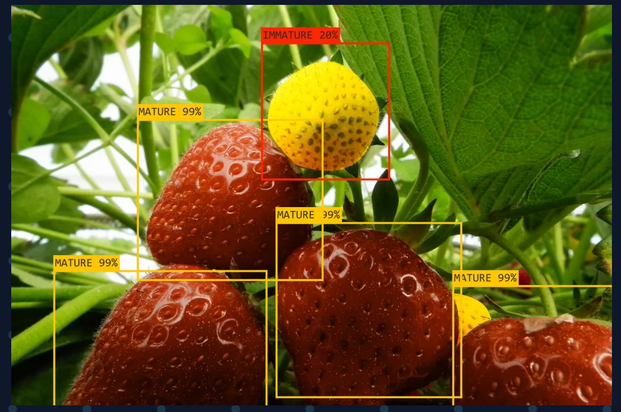


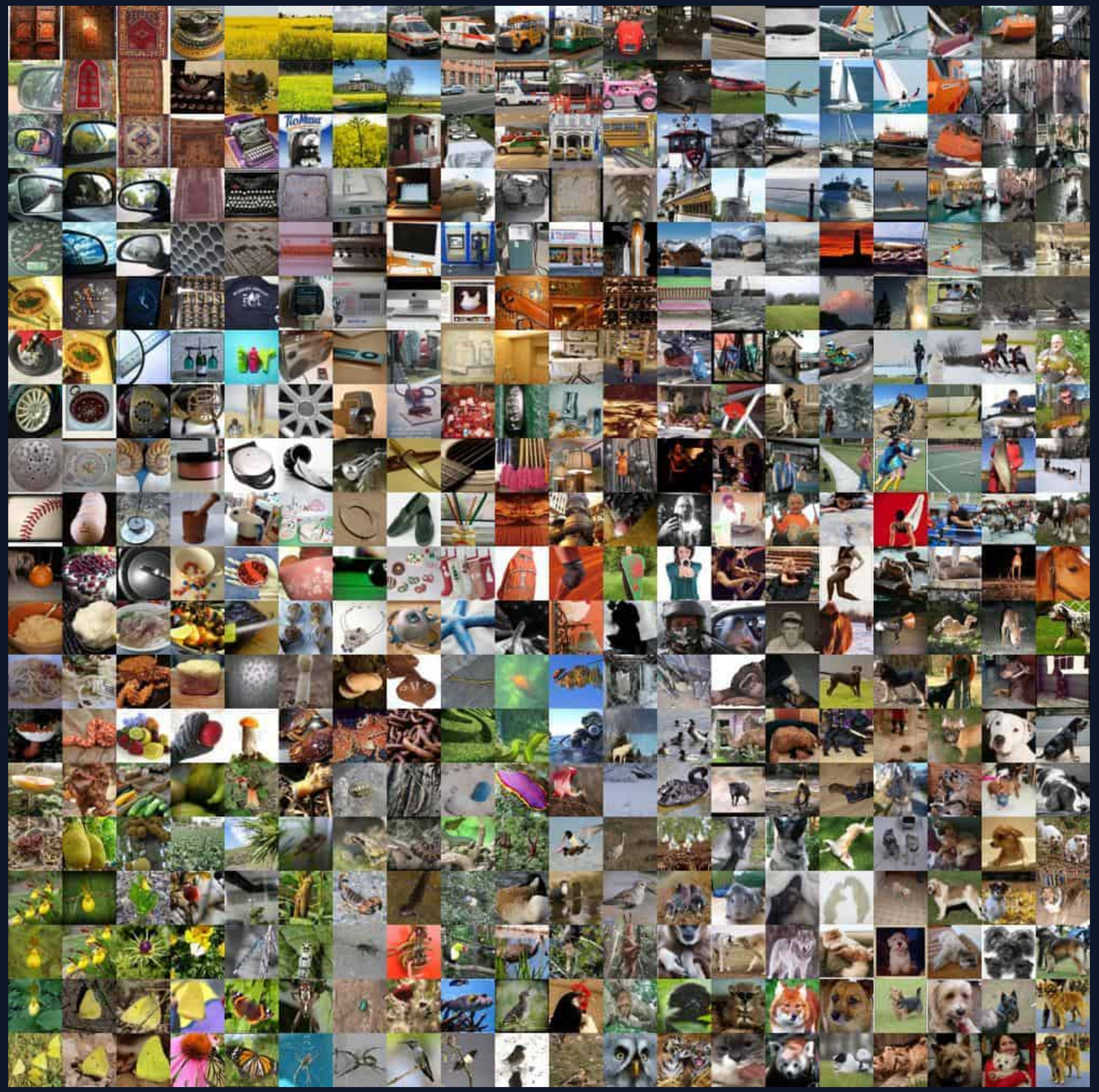
کشاورزی و صنعت

تشخیص آفت از پهپاد

کنترل کیفیت خط تولید

برداشت رباتیک



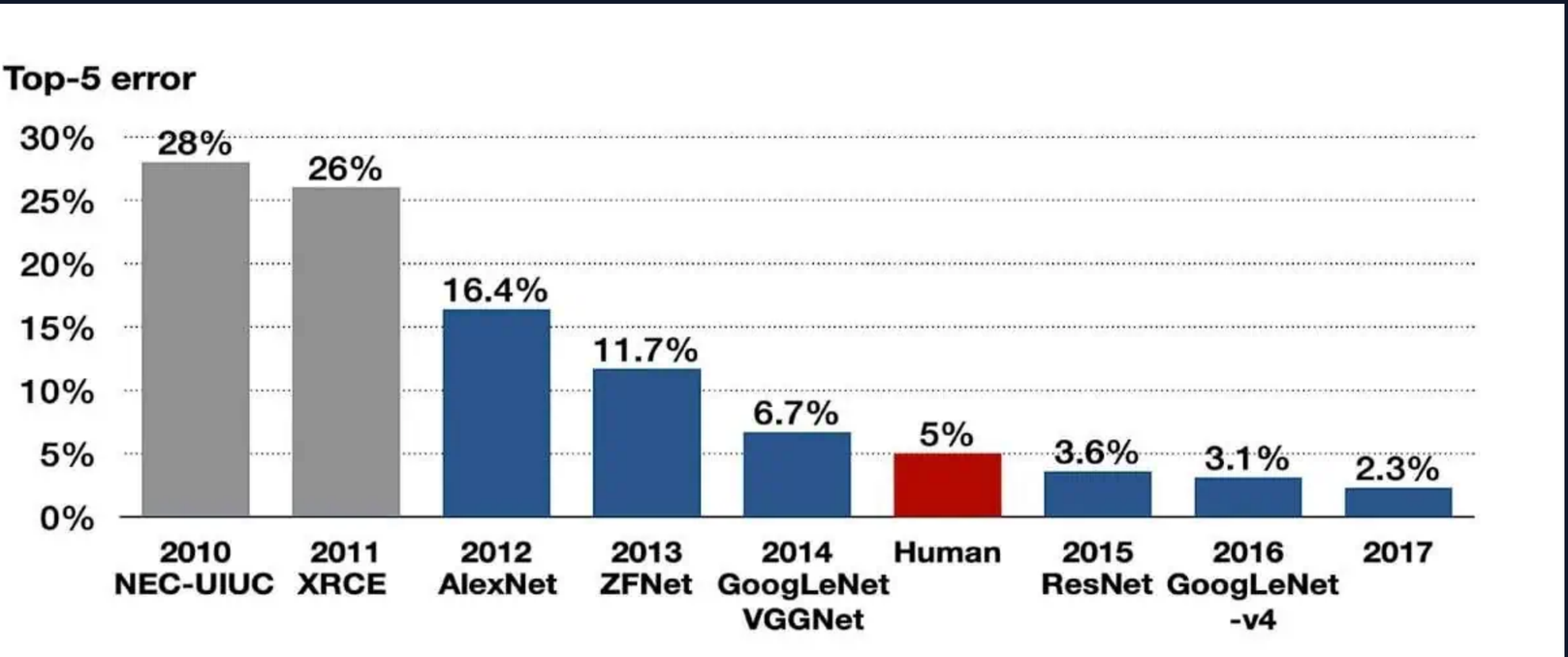


مسابقه‌ی ImageNet

- دسته‌بندی یک میلیون تصویر در ۱۰۰۰ کلاس

۲۰۱۲: سالی که ماشین‌ها بینا شدند

- سال‌ها خطای بهترین روش‌ها بالای ۲۶٪ بود و کم نمی‌شد
- AlexNet، یک CNN آموزش‌دیده روی دو GPU خانگی، خطا را یک‌باره به ۱۶.۴٪ رساند
- این لحظه، آغاز انفجار یادگیری عمیق بود، که هنوز ادامه دارد
- سه سال بعد: خطای ماشین از خطای انسان (حدود ۵٪) هم کمتر شد



https://viso.ai/deep-learning/imagenet/

داده‌های مرجع بینایی ماشین

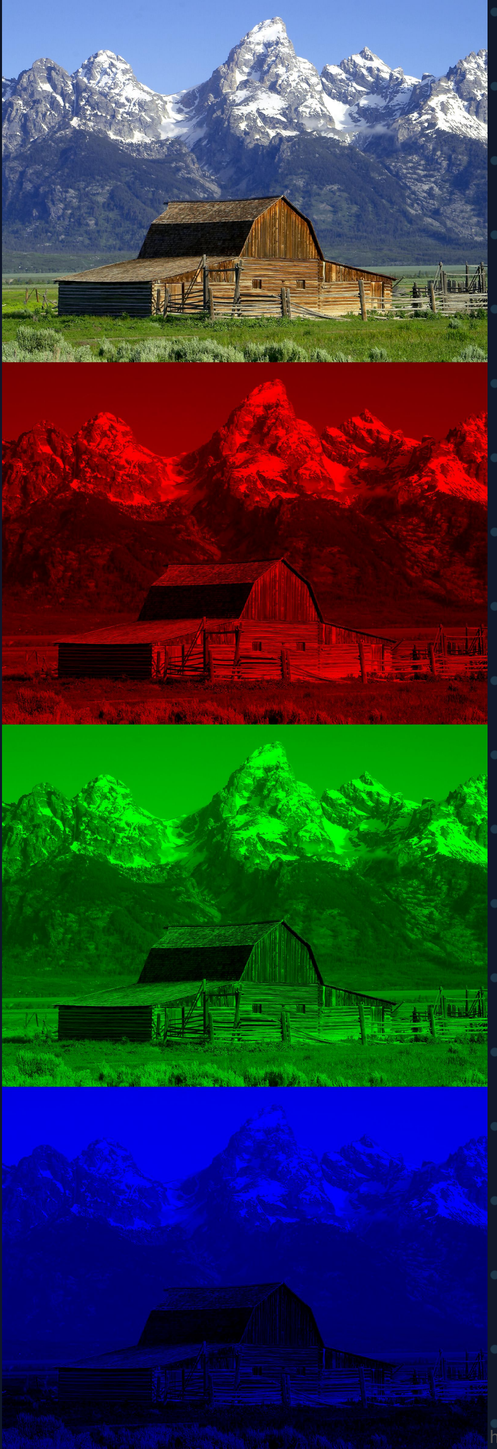
دیتاست	تصاویر	کلاس‌ها	ابعاد	نقش و کاربرد
MNIST	۷۰ هزار	۱۰	۲۸×۲۸ خاکستری	الفبای یادگیری، ارقام انگلیسی
Fashion-MNIST	۷۰ هزار	۱۰	۲۸×۲۸ خاکستری	جایگزین سخت‌تر MNIST روی لباس لباس‌ها
CIFAR-10	۶۰ هزار	۱۰	۳۲×۳۲ رنگی	اولین چالش واقعی رنگی، ۱۰ گروه اشیا
CIFAR-100	۶۰ هزار	۱۰۰	۳۲×۳۲ رنگی	همان تصاویر، ۱۰۰ گروه اشیا
ImageNet (ILSVRC)	۱٫۲۸ میلیون	۱۰۰۰	متغیر (~۲۲۴)	۱۰۰۰ گروه اشیای مختلف، مرجع اصلی
COCO	۳۳۰ هزار	۸۰ شیء	متغیر	تشخیص شیء، قطعه‌بندی، کپشن

سوال کلیدی

شما در این تصویر «گربه» می‌بینید؛

کامپیوتر چطور می‌تواند آن را ببیند و درک کند؟

https://en.wikiquote.org/wiki/Cats



تصویر از نگاه کامپیوتر: فقط عدد

- هر تصویر، جدولی از پیکسل‌هاست؛ هر پیکسل یک عدد روشنایی بین ۰ (سیاه) تا ۲۵۵ (سفید)
- تصویر رنگی: سه جدول روی هم — قرمز، سبز، آبی (RGB)
- یک عکس معمولی گوشی: ۳۰۰۰ × ۴۰۰۰ × ۳ = ۳۶ میلیون عدد!
- «دیدن» یعنی پیدا کردن الگو در این دریای عدد

https://en.wikipedia.org/wiki/RGB_color_model#/media/File:Barn_grand_tetons_rgb_separation.jpg

چرا شبکه‌ی معمولی برای تصویر کم می‌آورد؟

انفجار پارامترها

اتصال هر پیکسل به هر نورون:
۳۶ میلیون × ۱۰۰۰ نورون = ۳۶ میلیارد وزن!
فقط برای لایه‌ی اول

جابه‌جایی

اگر سر گربه یک پیکسل جابه‌جا شود،
همه‌ی ورودی‌ها عوض می‌شوند
انگار تصویری کاملا جدید است!

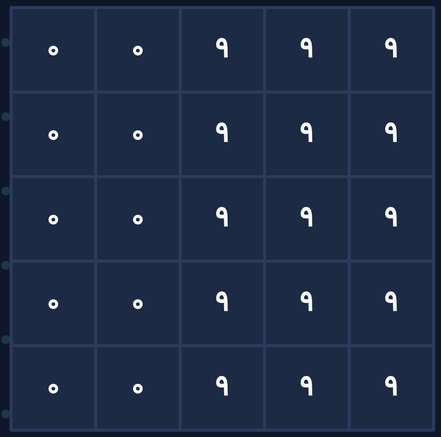
به ایده‌ای هوشمندانه‌تر نیاز داریم: الهام از چشم و مغز

ایده‌ی کلیدی: کانولوشن (Convolution)

- به‌جای نگاه به کل تصویر، با یک «ذره‌بین کوچک» (فیلتر) روی تصویر بلغز
- فیلتر: جدولی کوچک از وزن‌ها، مثلا ۳×۳، که دنبال یک الگوی خاص در نقطه به نقطه‌ی تصویر ورودی می‌گردد
- در هر نقطه، فیلتر با آن تکه‌ی تصویر مقایسه می‌شود؛ شباهت زیاد = خروجی بزرگ
- خروجی: «نقشه‌ی ویژگی»، کجاهای تصویر این الگو را دارند؟
- نکته‌ی طلایی: وزن‌های فیلتر از پیش تعیین نمی‌شوند؛ خود شبکه آن‌ها را یاد می‌گیرد!

شروع

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

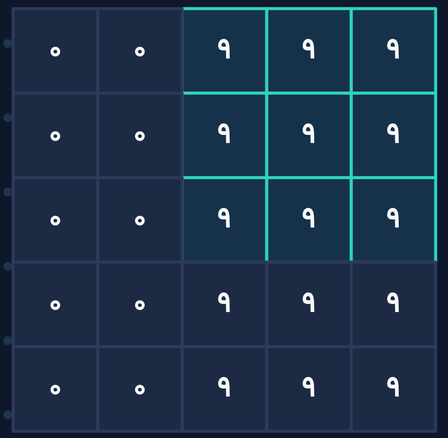
=

نقشه‌ی ویژگی ۳×۳

قانون بازی

پنجره‌ی ۳×۳ از گوشه‌ی بالا-راست شروع می‌کند، ستون‌به‌ستون می‌لغزد و برای هر ایست، یک عدد می‌سازد

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

خانه‌ی (ردیف ۱، ستون ۱):

هر ردیفِ پنجره:

۹×(-۱) + ۹×۰ + ۹×(+۱) = ۰

سه ردیف: ۰ × ۳ = ۰

کانولوشن، قاب‌به‌قاب

تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

خانه‌ی (ردیف ۱، ستون ۲):

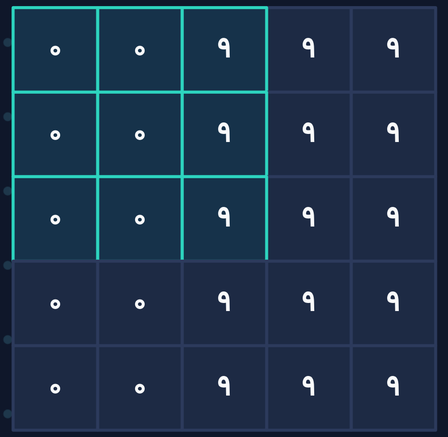
هر ردیفِ پنجره:

۹×(+۱) + ۹×۰ + ۰×(-۱) = ۹

سه ردیف: ۹ × ۳ = ۲۷

قاب ۳ از ۹

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

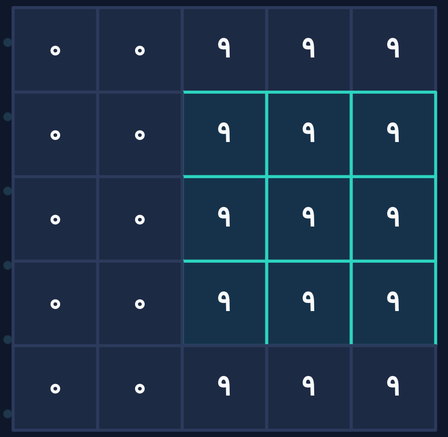
خانه‌ی (ردیف ۱، ستون ۳):

هر ردیفِ پنجره:

۹×(+۱) + ۰×۰ + ۰×(-۱) = ۹

سه ردیف: ۹ × ۳ = ۲۷

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

خانه‌ی (ردیف ۲، ستون ۱):

هر ردیفِ پنجره:

۹×(+۱) + ۹×۰ + ۹×(-۱) = ۰

سه ردیف: ۰ × ۳ = ۰

کانولوشن، قاب‌به‌قاب

تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

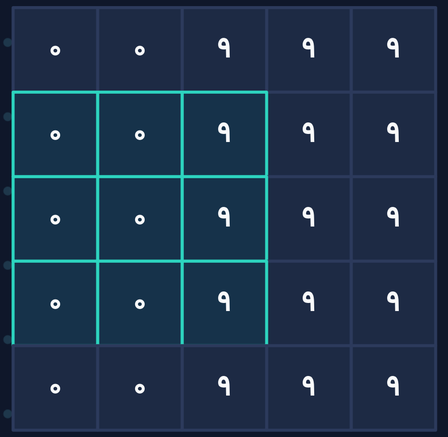
خانه‌ی (ردیف ۲، ستون ۲):

هر ردیفِ پنجره:

۹×(+۱) + ۹×۰ + ۰×(-۱) = ۹

سه ردیف: ۹ × ۳ = ۲۷

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

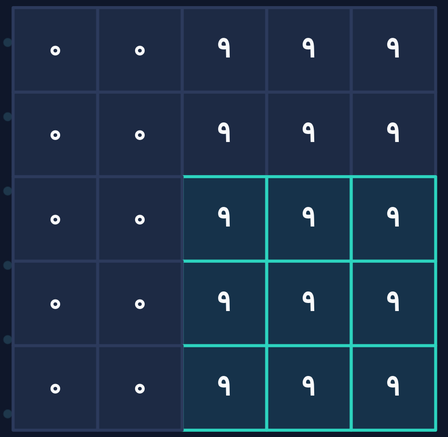
خانه‌ی (ردیف ۲، ستون ۳):

هر ردیفِ پنجره:

۹×(+۱) + ۰×۰ + ۰×(-۱) = ۹

سه ردیف: ۹ × ۳ = ۲۷

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

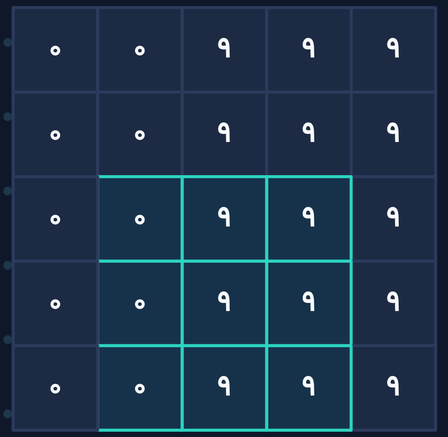
خانه‌ی (ردیف ۳، ستون ۱):

هر ردیفِ پنجره:

۹×(+۱) + ۹×۰ + ۹×(-۱) = ۰

سه ردیف: ۰ × ۳ = ۰

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

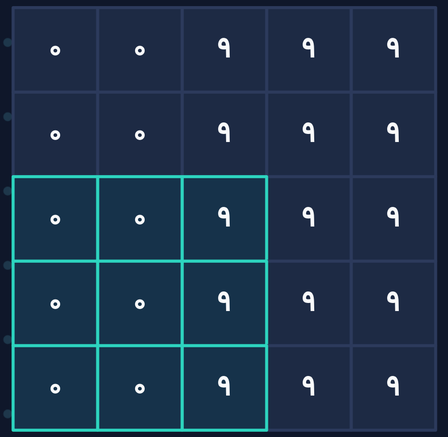
خانه‌ی (ردیف ۳، ستون ۲):

هر ردیفِ پنجره:

۹×(+۱) + ۹×۰ + ۰×(-۱) = ۹

سه ردیف: ۹ × ۳ = ۲۷

کانولوشن، قاب‌به‌قاب



تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=

نقشه‌ی ویژگی ۳×۳

خانه‌ی (ردیف ۳، ستون ۳):

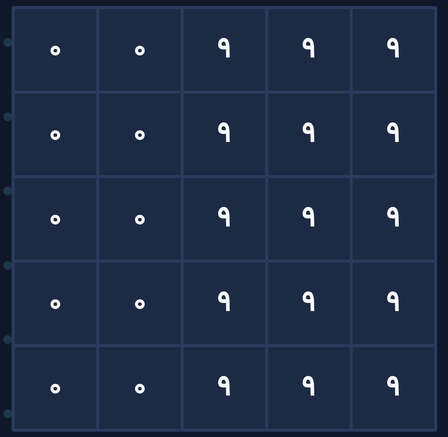
هر ردیفِ پنجره:

۹ = (۱-)×۰ + ۰×۰ + (۱+)×۹

سه ردیف: ۹ × ۳ = ۲۷

کانولوشن، قاب‌به‌قاب

پایان!

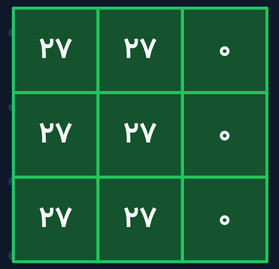


تصویر ورودی ۵×۵

*

فیلتر ۳×۳

=



نقشه‌ی ویژگی ۳×۳

۹ ایست، ۹ عدد

ستون ۲۷ها = ردپای لبه

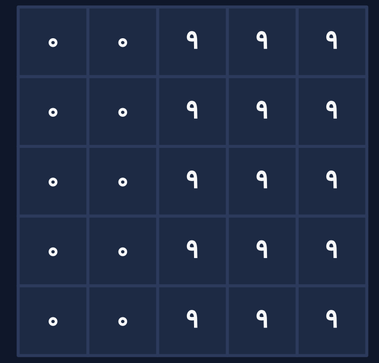
ستون ۰ها = ناحیه‌ی یکنواخت

نقشه‌ی ویژگی کامل شد!

در شبکه‌ی واقعی، این فرایند برای ده‌ها فیلتر و در همه‌ی لایه‌ها هم‌زمان اجرا می‌شود.

فیلترهای مختلف، چشم‌های مختلف

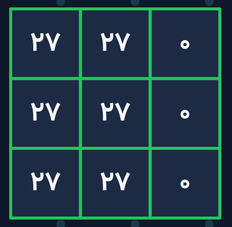
همان تصویر، دو فیلتر متفاوت — هر فیلتر فقط الگوی خودش را «می‌بیند»:



تصویر ورودی (لبه‌ی عمودی دارد)

=

فیلتر عمودی‌یاب



لبه پیدا شد! (ستون ۲۷ها)

=

فیلتر افقی‌یاب

همه ۰ — لبه‌ی افقی وجود ندارد

یک لایه‌ی کانولوشنی ده‌ها فیلتر دارد؛ هر فیلتر یک «آشکارساز الگو»ی مستقل است و نقشه‌ی خودش را می‌سازد.

و فراموش نکنید: این اعداد را ما ننوشته‌ایم — گرادیان کاهشی آن‌ها را از داده پیدا می‌کند.

سه خاصیت طلایی کانولوشن

وزن مشترک

یک فیلتر برای کل تصویر:

۹ وزن به‌جای میلیاردها

نگاه محلی

هر ویژگی از همسایه‌های

نزدیکش ساخته می‌شود

مقاوم به جابه‌جایی

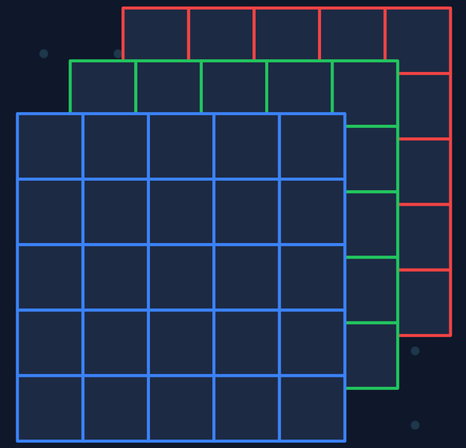
گربه هر جای قاب باشد،

همان فیلتر پیدایش می‌کند

هر لایه ده‌ها فیلتر مختلف دارد؛ هر فیلتر دنبال الگوی خودش می‌گردد.

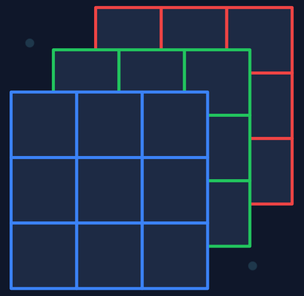
عمق ورودی: تصویر رنگی سه کانال دارد

تصویر رنگی = سه جدول روی هم (قرمز، سبز، آبی) — فیلتر هم باید سه‌لایه باشد: ۳×۳×۳



سه کانال ورودی ۵×۵ (RGB)

*



یک فیلتر ۳×۳×۳ (۲۷ وزن + بایاس)

=

فقط یک نقشه‌ی ویژگی!

قاعده‌ی طلایی: عمق فیلتر همیشه برابر عمق ورودی است
خروجی یک فیلتر همیشه تک‌لایه است

کانولوشن سه‌کاناله، قاب‌به‌قاب

شروع

صحنه: غروب دریا

آسمانِ قرمز بالا، دریای آبی پایین،
مهِ سبزِ یکنواخت همه‌جا

فیلترِ ما «افق‌یاب» است:

لایه‌ی قرمز: روشناییِ بالا - پایین

لایه‌ی سبز: صفر — بی‌اهمیت

لایه‌ی آبی: روشناییِ پایین - بالا

نقشه‌ی ویژگی (تک‌لایه!)

سه لایه‌ی فیلتر — هر لایه متفاوت!

کانولوشن سه‌کاناله، قاب‌به‌قاب

خانه‌ی (ردیف ۱، ستون ۱):

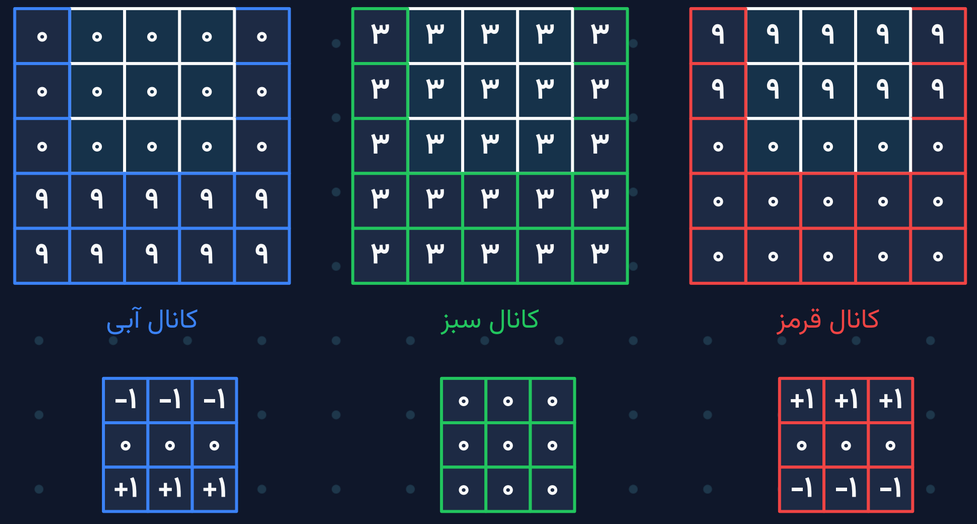
قرمز: (۹ − ۰) × ۳ = ۲۷

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۰ − ۰) × ۳ = ۰

جمع: ۲۷ + ۰ + ۰ = ۲۷

کانولوشن سه‌کاناله، قاب‌به‌قاب



سه لایه‌ی فیلتر — هر لایه متفاوت!

نقشه‌ی ویژگی (تک‌لایه!)

خانه‌ی (ردیف ۱، ستون ۲):

قرمز: (۹ − ۰) × ۳ = ۲۷

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۰ − ۰) × ۳ = ۰

جمع: ۲۷ + ۰ + ۰ = ۲۷

کانولوشن سه‌کاناله، قاب‌به‌قاب

خانه‌ی (ردیف ۱، ستون ۳):

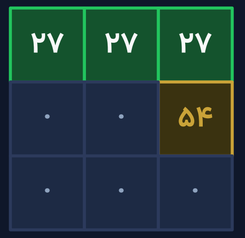
قرمز: (۹ − ۰) × ۳ = ۲۷

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۰ − ۰) × ۳ = ۰

جمع: ۲۷ + ۰ + ۰ = ۲۷

کانولوشن سه‌کاناله، قاب‌به‌قاب



خانه‌ی (ردیف ۲، ستون ۱):

قرمز: (۹ − ۰) × ۳ = ۲۷

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۹ − ۰) × ۳ = ۲۷

جمع: ۲۷ + ۰ + ۲۷ = ۵۴

کانولوشن سه‌کاناله، قاب‌به‌قاب

سه لایه‌ی فیلتر — هر لایه متفاوت!

نقشه‌ی ویژگی (تک‌لایه!)

خانه‌ی (ردیف ۲، ستون ۲):

قرمز: (۹ − ۰) × ۳ = ۲۷

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۹ − ۰) × ۳ = ۲۷

جمع: ۲۷ + ۰ + ۲۷ = ۵۴

کانولوشن سه‌کاناله، قاب‌به‌قاب

خانه‌ی (ردیف ۲، ستون ۳):

قرمز: (۹ − ۰) × ۳ = ۲۷

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۹ − ۰) × ۳ = ۲۷

جمع: ۲۷ + ۰ + ۲۷ = ۵۴

کانولوشن سه‌کاناله، قاب‌به‌قاب

سه لایه‌ی فیلتر — هر لایه متفاوت!

نقشه‌ی ویژگی (تک‌لایه!)

خانه‌ی (ردیف ۳، ستون ۱):

قرمز: (۰ − ۰) × ۳ = ۰

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۹ − ۰) × ۳ = ۲۷

جمع: ۰ + ۰ + ۲۷ = ۲۷

کانولوشن سه‌کاناله، قاب‌به‌قاب

سه لایه‌ی فیلتر — هر لایه متفاوت!

نقشه‌ی ویژگی (تک‌لایه!)

خانه‌ی (ردیف ۳، ستون ۲):

قرمز: (۰ − ۰) × ۳ = ۰

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۹ − ۰) × ۳ = ۲۷

جمع: ۰ + ۰ + ۲۷ = ۲۷

کانولوشن سه‌کاناله، قاب‌به‌قاب

سه لایه‌ی فیلتر — هر لایه متفاوت!

نقشه‌ی ویژگی (تک‌لایه!)

خانه‌ی (ردیف ۳، ستون ۳):

قرمز: (۰ − ۰) × ۳ = ۰

سبز: همه‌ی وزن‌ها ۰ ← نادیده!

آبی: (۹ − ۰) × ۳ = ۲۷

جمع: ۰ + ۰ + ۲۷ = ۲۷

کانولوشن سه‌کاناله، قاب‌به‌قاب

پایان!

افق پیدا شد!

با ترکیب آزاد وزن‌ها در عمق، فیلتر می‌تواند هر «الگوی رنگی-مکانی» را آشکار کند: چمن، پوست، آتش، افق...

عمق خروجی: هر فیلتر، یک ورق

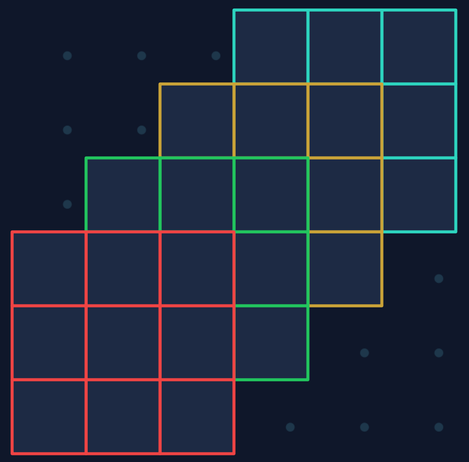
چهار فیلتر متفاوت روی همان ورودی ← چهار نقشه‌ی ویژگی، روی هم = خروجی با عمق ۴

فیلتر ۱: لبه‌ی عمودی

فیلتر ۲: لبه‌ی افقی

فیلتر ۳: قطر

فیلتر ۴: لکه‌ی رنگ



پشته‌ی چهار نقشه‌ی ویژگی — عمق خروجی = ۴

حساب سرانگشتی

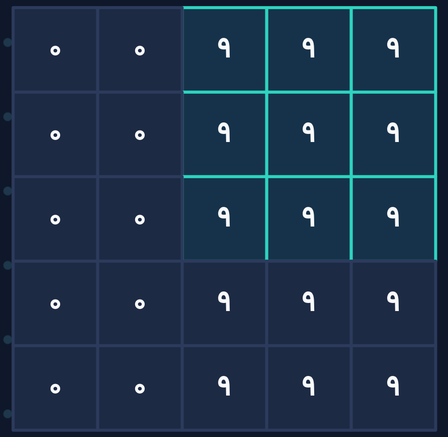
ورودی: ۵×۵×۳

۴ فیلتر ۳×۳×۳

خروجی: ۳×۳×۴

پارامترها: ۴×(۲۷+۱) = ۱۱۲

پنجره‌ی جهنده: Stride = ۲



تصویر ورودی ۵×۵

فیلتر ۳×۳

خروجی فقط ۲×۲!

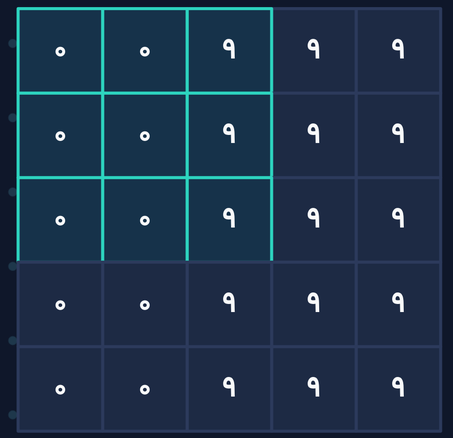
گام (Stride) = ۲

شروع از گوشه (ردیف ۱، ستون ۱)

حساب همان است: نتیجه ۰

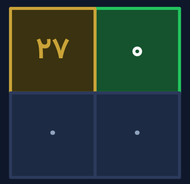
اما قدمِ بعد فرق دارد...

پنجره‌ی جهنده: Stride = ۲



تصویر ورودی ۵×۵

فیلتر ۳×۳



خروجی فقط ۲×۲!

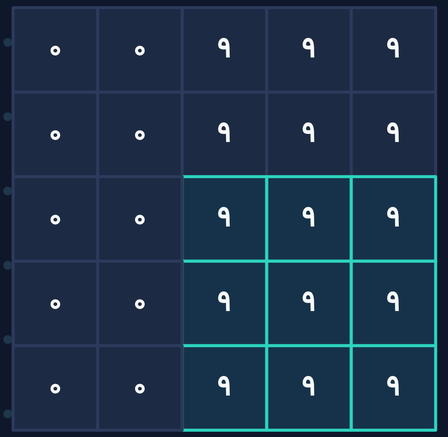
گام (Stride) = ۲

پرش دوتایی به ستون ۳!

ستون ۲ کلا رد شد

نتیجه: ۲۷

پنجره‌ی جهنده: Stride = ۲



تصویر ورودی ۵×۵

فیلتر ۳×۳

خروجی فقط ۲×۲!

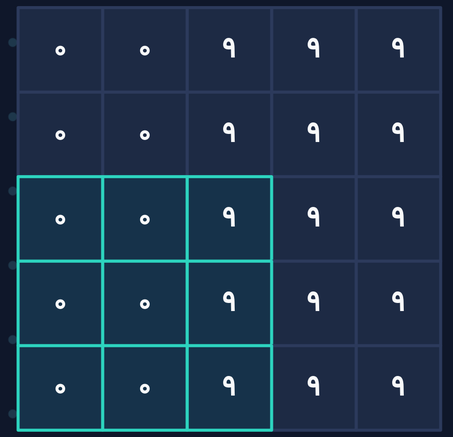
گام (Stride) = ۲

پرش عمودی دوتایی به ردیف ۳

ردیف ۲ هم رد شد

نتیجه: ۰

پنجره‌ی جهنده: Stride = ۲



تصویر ورودی ۵×۵

فیلتر ۳×۳

خروجی فقط ۲×۲!

گام (Stride) = ۲

آخرین ایست: (ردیف ۳، ستون ۳)

نتیجه: ۲۷

چهار ایست، تمام!

کوچک‌سازی هوشمندانه :Pooling

Max Pooling چیست؟

از هر ناحیه‌ی ۲×۲ فقط بزرگ‌ترین عدد می‌ماند
ابعاد تصویر نصف می‌شود، مهم‌ترین سیگنال حفظ می‌شود
«این ویژگی این حوالی بود». دقیقا کجا؟ مهم نیست

چرا به درد می‌خورد؟

محاسبات لایه‌های بعدی سبک‌تر می‌شود
مقاومت به جابه‌جایی‌های کوچک بیشتر می‌شود
دید فیلترهای بعدی وسیع‌تر می‌شود

Max Pooling

ورودی ۴×۴ با پنجره‌ی ۲×۲

ورودی ۴×۴: چهار ناحیه‌ی رنگی

خروجی ۲×۲: بیشینه‌ی هر ناحیه

ناحیه‌ی آبی: بیشینه‌ی {۱، ۳، ۵، ۶} = ۶ ناحیه‌ی قرمز: بیشینه‌ی {۸ ،۰ ،۴ ،۹} = ۹

بدون حاشیه: آب‌رفتن اندازه‌ی تصویر

سه لایه‌ی کانولوشن ۳×۳ پشت‌سرهم، بدون هیچ حاشیه‌ای (P=۰) — تماشا کنید چه می‌ماند:

هر لایه دو پیکسل می‌خورَد (F - ۱ = ۲)؛ بعد از سه لایه از ۴۹ پیکسل فقط «یکی» ماند!

با شبکه‌های ده‌ها لایه‌ای، بدون حاشیه اصلا تصویری باقی نمی‌ماند که پردازش شود.

مشکل پنهان: گوشه‌ها کمتر دیده می‌شوند

عدد هر خانه: این پیکسل در چند پنجره‌ی ۳×۳ ظاهر می‌شود؟ (بدون حاشیه، روی تصویر ۵×۵)

۱	۲	۳	۲	۱
۲	۴	۶	۴	۲
۳	۶	۹	۶	۳
۲	۴	۶	۴	۲
۱	۲	۳	۲	۱

تعداد پنجره‌های شاملِ هر پیکسل

بی‌عدالتی آشکار!

پیکسل مرکزی در ۹ پنجره دیده می‌شود

پیکسل گوشه فقط در ۱ پنجره!

یعنی اطلاعات لبه‌ها و گوشه‌ها
سهم بسیار کمتری در خروجی دارند

Padding: حاشیه‌ی صفر دور تصویر

همان تصویر ۵×۵ با یک دور حاشیه‌ی صفر (P=۱) — حالا ۷×۷ شده است:

۰	۰	۰	۰	۰	۰	۰
۰	۰	۰	۹	۹	۹	۰
۰	۰	۰	۹	۹	۹	۰
۰	۰	۰	۹	۹	۹	۰
۰	۰	۰	۹	۹	۹	۰
۰	۰	۰	۹	۹	۹	۰
۰	۰	۰	۰	۰	۰	۰

خانه‌های کم‌رنگ: صفرهای اضافه‌شده

بدون حاشیه، خروجی هر بار کوچک می‌شود: ۵ ← ۳ ← ۱

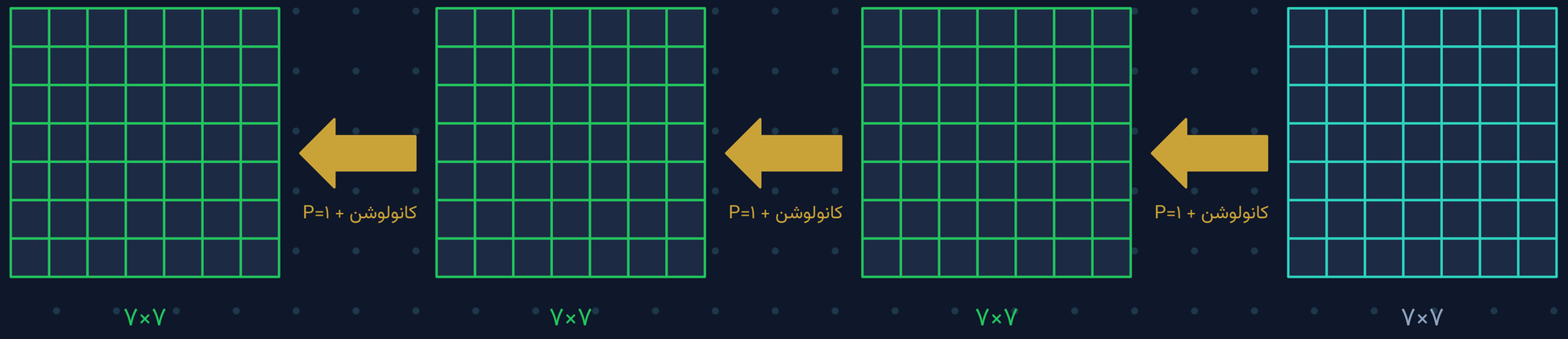
پیکسل‌های گوشه فقط در یک پنجره دیده می‌شوند — به آن‌ها «کم‌محلی» می‌شود!

با P=۱ و فیلتر ۳×۳: خروجی هم‌اندازه‌ی ورودی می‌ماند

قرارداد رایج: آن‌قدر حاشیه بده که اندازه‌ی تصویر، در خروجی حفظ شود؛ کوچک‌کردن را به عهده‌ی Pooling یا Stride بگذار.

با حاشیه: اندازه، دست‌نخورده

همان سه لایه، این‌بار هر بار با یک دور حاشیه‌ی صفر (P=۱) پیش از کانولوشن:



معماری کامل: شبکه‌ی کانولوشنی (CNN)

قطعه‌ها را کنار هم بچینیم — از پیکسل خام تا برچسب نهایی:

یک CNN کامل: ابعاد و پارامترهای هر لایه

لایه	ابعاد خروجی	پارامترها
ورودی (رقم دست‌نویس)	۲۸×۲۸×۱	۰
کانولوشن ۳×۳، ۸ فیلتر، P=۱	۲۸×۲۸×۸	۸۰
MaxPool ۲×۲	۱۴×۱۴×۸	۰
کانولوشن ۳×۳، ۱۶ فیلتر، P=۱	۱۴×۱۴×۱۶	۱,۱۶۸
MaxPool ۲×۲	۷×۷×۱۶	۰
تمام‌متصل به ۱۰ + Flatten	۱۰	۷,۸۵۰

جمع کل: حدود ۹ هزار پارامتر — و روی MNIST به دقت ~۹۹٪ می‌رسد!

سلسله‌مراتب دیدن: چه چیزی یاد گرفته می‌شود؟

لایه‌های اول

لبه‌ها، خط‌ها،

لکه‌های رنگ

لایه‌های میانی

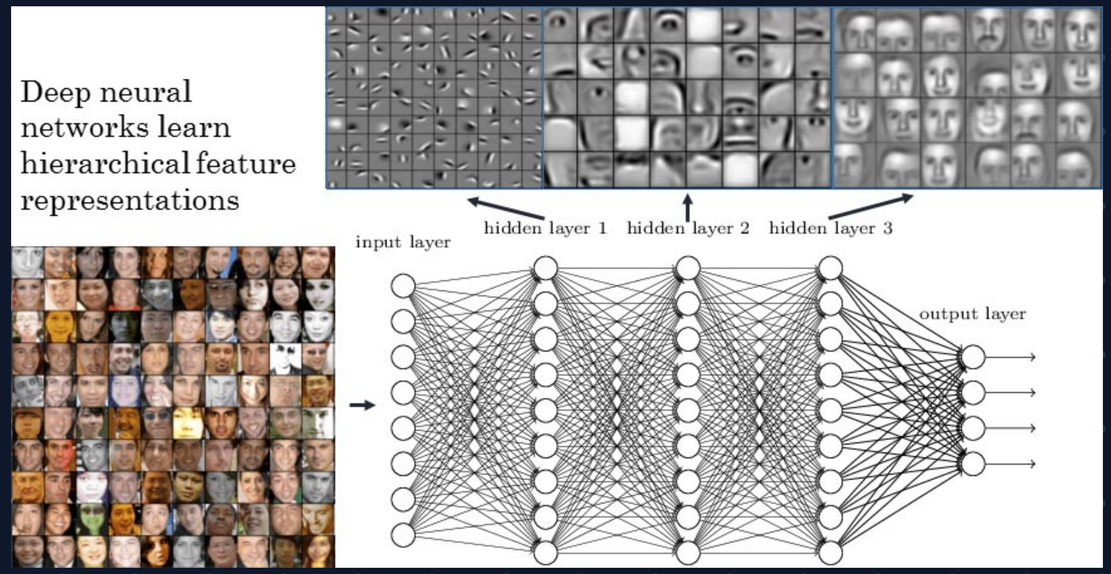
بافت‌ها و اجزا:

خز، چشم، گوش، چرخ

لایه‌های آخر

اشیای کامل:

گربه، خودرو، چهره‌ی شما



https://www.rsipvision.com/exploring-deep-learning/

افزون‌سازی داده: از هر تصویر، ده‌ها تصویر

گربه‌ی آینه‌شده هنوز گربه است — مدل باید به «ذات» نگاه کند، نه به قاب:

هندسی

آینه‌ی افقی، چرخش کوچک،

برش تصادفی، بزرگ‌نمایی

رنگی و نوری

روشنایی، کنتراست،

اعوجاج رنگ، تاری (Blur)

پیشرفته

Cutout: حذف تکه‌ای تصادفی

Mixup: ترکیب دو تصویر و برچسب‌ها

افزون‌سازی در عمل: کتابخانه‌ی albumentations

استاندارد حرفه‌ای‌ها (و برنده‌های Kaggle) — چند خط کد، یک خط تولید کامل:

```
import albumentations as A

transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=15, p=0.7),
    A.RandomBrightnessContrast(p=0.4),
    A.GaussianBlur(p=0.2),
    A.CoarseDropout(max_holes=8, p=0.3),
    A.RandomCrop(224, 224),
])

augmented = transform(image=img)["image"]
```

هر p یعنی «احتمال اجرا»: HorizontalFlip در نیمی از دفعات اعمال می‌شود

یعنی چرخش تصادفی بین -۱۵ تا +۱۵ درجه limit=۱۵

است: سوراخ‌های تصادفی Cutout همان CoarseDropout

ترتیب مهم است: اول تغییرها، آخر برش به اندازه‌ی ورودی مدل

برای تشخیص شیء، جعبه‌ها را هم خودش هماهنگ جابه‌جا می‌کند!